🌐 Web Development Lab

Our Services, CSS Grid— Practical Session Guide

15 Guided Steps • Beginner–Intermediate • Estimated Time: 50 mins

Lab Overview

In this session you will build a **Solution Section using CSS Grid** — a structured layout that displays multiple cards in a clean and responsive format.

You will learn how to:

- Use **CSS Grid for layout**
- Control columns and spacing
- Structure content into reusable UI components (cards)

Each step builds progressively so you can **observe how Grid transforms layout behavior in real-time**.

📌 **Note: Continue working in your existing `index.html` file.**

Prerequisites

- ☑ Completed Navbar Lab
- ☑ Completed Hero Section Lab
- ☑ Basic understanding of Flexbox
- ☑ Basic CSS knowledge

1. Add the Solution Section HTML

Objective: Create the structure of the solution section.

⏱ 5 min

Background

This section introduces a **new layout pattern: cards inside a container**.

We group elements using `<section>` and `<div>` for organization. Each card represents a **feature or offering**.

Instructions

1. Add the HTML below your Hero Section

Code Snippet

```
<section class="solutionSection">
    <div class="SolutionTitle">
        <h2>Our Solution</h2>
        <small>Unlocking Field Documentation Potential Through Comprehensive
Digital Tools</small>
    </div>

    <div class="cards">
        <div class="cardOne">
            <h3>Real Time Reporting</h3>
            <p>Capture and document field conditions instantly...</p>
        </div>

        <div class="cardTwo">
            <h3>Quality Assurance</h3>
            <p>Maintain precise documentation standards...</p>
        </div>

        <div class="cardThree">
            <h3>Quality Assurance</h3>
            <p>Maintain precise documentation standards...</p>
        </div>
    </div>
</section>
```

💡 **Tip: Cards are reusable UI components used in almost every modern website.**

2. Style the Section Container

Objective: Center and space the section properly.

Background

This section controls **overall layout spacing**.

1. **margin: auto;**
    - Centers the section horizontally
2. **margin: 100px 82px;**
    - Adds vertical (100px) and horizontal (82px) spacing
3. **align-items: center;**
    - Used for alignment (though effective mainly in flex/grid contexts)

Instructions

1. Style `.solutionSection`

Code Snippet

```
.solutionSection{
  align-items: center;
  margin: auto;
  margin: 100px 82px;
}
```

💡 **Tip: Spacing creates visual separation between sections.**

3. Center the Section Title

Objective: Align heading and subtitle centrally.

⏱ 4 min

Background

1. **text-align: center;**
    - Centers inline content like text inside a container

Instructions

1. Style `.SolutionTitle`

Code Snippet

```
.SolutionTitle{
  text-align: center;
}
```

💡 **Tip: Section headers are usually centered for emphasis.**

4. Convert Cards Container into Grid

Objective: Enable Grid layout.

Background

CSS Grid is a **two-dimensional layout system**.

1. **display: grid;**
    - Turns `.cards` into a grid container
2. **grid-template-columns: repeat(3, auto);**
    - Creates 3 columns, Each column takes automatic width

Instructions

1. Apply Grid layout

Code Snippet

```
.cards{
  display:grid;
  grid-template-columns: repeat(3,auto);
}
```

💡 **Tip: Grid is best for layouts involving rows AND columns.**

5. Understand Grid Columns

Objective: Learn how columns behave.

Background

1. repeat(3, auto);

- `3` → number of columns
- `auto` → width based on content

This means:

- 3 cards = 3 columns
- Each adjusts automatically

Instructions

1. Observe how cards align

💡 **Tip: You can also use `1fr` instead of `auto` for equal widths.**

6. Add Spacing Between Grid Items

Objective: Improve spacing between cards.

Background

1. **gap: 32px**;
    - Adds space between grid items (rows and columns)

Instructions

1. Add gap

Code Snippet

```
.cards{
  display:grid;
  grid-template-columns: repeat(3,auto);
  gap: 32px;
}
```

💡 **Tip: gap replaces margin hacks — cleaner and modern.**

7. Add Margin Above Cards

Objective: Separate title from cards.

Background

1. **margin: 64px 0px 0px;**
    - This means apply 0 margin to Left/right while margin = 64px

Instructions

1. Add margin

Code Snippet

```
.cards{
  display:grid;
  grid-template-columns: repeat(3,auto);
  gap: 32px;
  margin: 64px 0px 0px;
}
```

💡 **Tip: Always separate sections visually.**

8. Style Individual Cards

Objective: Give cards structure and appearance.

Background

1. **background-color: #FFFFFF;**
    - Sets card background
2. **border: 1px solid;**
    - Adds border
3. **padding: 0px 32px 32px;**
    - Adds internal spacing

📌 ***Note: You can apply css values to more than one element at one time by calling all of them at the same time as can be seen below, CardOne, .CardTwo, CardThree all have the same CSS properties. This is called "Group Selectors"***

Instructions

1. Style all cards

Code Snippet

```
.cardOne, .cardTwo, .cardThree{
  background-color: #FFFFFF;
  border: 1px solid;
  border-color: white;
  text-align: center;
  padding: 0px 32px 32px;
}
```

💡 **Tip: Group selectors reduce repetition.**

9. Style Card Headings

Objective: Improve typography of card titles.

Background

1. **font-size: 15px;**
    - Controls text size
2. **line-height: 25.5px;**
    - Controls spacing between lines
3. **font-family: …**
    - Defines typography style

Instructions

1. Style `<h3> tags in the cards class using group selector`

Code Snippet

```
.cardOne h3, .cardTwo h3, .cardThree h3{
  line-height: 25.5px;
  font-size: 15px;
  font-family: 'Lucida Sans', 'Verdana', sans-serif;
}
```

💡 **Tip: Typography consistency improves UI quality.**

10. Center Card Content

Objective: Align content properly inside cards.

Background

1. **text-align: center;**
    - Centers text inside cards

Instructions

1. Ensure alignment

Code

```
.cardOne, .cardTwo, .cardThree{
 background-color: #FFFFFF;
 border: 1px solid;
 border-color: white;
 text-align: center;
}
```

💡 **Tip: Cards often use centered content for balance.**

11. Understand Grid vs Flexbox

Objective: Reinforce layout concepts.

Background

- Flexbox : 1D layout (row OR column)
- Grid : 2D layout (rows AND columns)

Here:

- Grid controls overall layout
- Flexbox can be used inside cards if needed

💡 **Tip: Use Grid for structure, Flexbox for alignment.**

12. Improve Visual Spacing

Objective: Enhance readability and layout balance.

Background

Spacing is controlled using:

- margin (outside)
- padding (inside)
- gap (between elements)

Instructions

1. Review spacing

💡 **Tip: Good spacing = professional UI.**

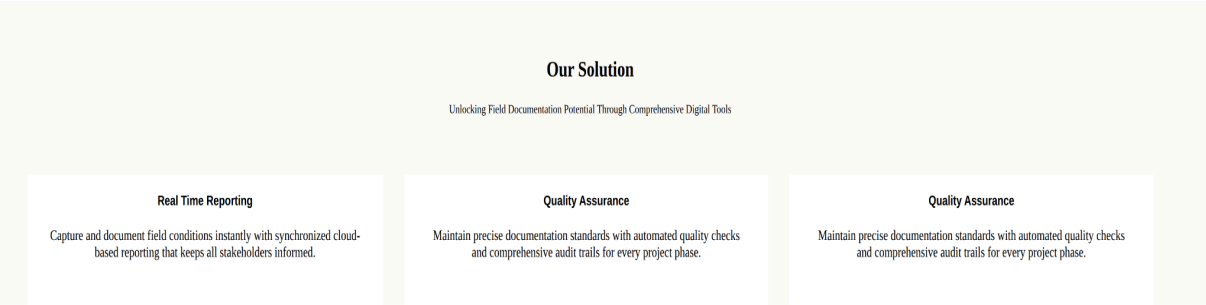
Final Review and Testing

Objective: Validate complete section.

Instructions

1. Refresh browser
2. Check alignment and spacing
3. Ensure cards display correctly

Final Output



Final Check

- Grid layout working correctly
- Cards aligned properly
- No CSS errors